SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Assessment Tool 1 – Prompt Engineering Challenge

| | |
|---|---|
| **Name:** Joanathan Packia Singh | **Reg. No.:** 192472229 |
| **Course Code:** CSA6502 | **Course Title:** Generative AI and Large Language Models |

## Q12. Least-to-Most Prompting

*Question: Write a prompt that decomposes a complex task (e.g., "plan a 3-day trip on a $500 budget") into an ordered list of sub-problems, solves each one sequentially, and combines the sub-answers into a final solution.*

### Prompt

```
You are a planning assistant that uses Least-to-Most prompting.
Task: Plan a 3-day trip on a $500 total budget.

Step 1 - Decompose the task into an ordered list of sub-problems
(e.g., destination choice, transport cost, lodging cost, food
budget, activities budget).

Step 2 - Solve each sub-problem ONE AT A TIME, in order, using the
answer from each previous step as context for the next.

Step 3 - After all sub-problems are solved, combine the results
into a final day-by-day itinerary with a running budget total
that does not exceed $500.

Show your work for every step before giving the final combined plan.
```

### Simple Grok-LLM Command (for reference / execution)

```
curl https://api.x.ai/v1/chat/completions \
  -H "Authorization: Bearer $GROK_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
     "model": "grok-4",
    "messages": [
       {"role": "user", "content": "<paste the prompt above>"}
    ]
  }'
```

### Sample Model Output (running the prompt)

```
Sub-problems identified:
1. Destination choice  2. Transport cost  3. Lodging cost
4. Food budget         5. Activities budget

Step 1 - Destination: A nearby city reachable by bus/train
(keeps transport cheap). Chosen: a city 3 hrs away.

Step 2 - Transport: Round-trip bus ticket = $60 total.
Running total: $60 / $500.

Step 3 - Lodging: 2 nights in a budget hostel/guesthouse
@ $45/night = $90. Running total: $150 / $500.

Step 4 - Food: 3 days @ $25/day (breakfast, lunch, dinner)
= $75. Running total: $225 / $500.

Step 5 - Activities: Museum pass ($20), walking tour ($15),
one paid attraction ($40) = $75. Running total: $300 / $500.

Final Combined Itinerary:
Day 1: Travel in (bus $60), check into hostel ($45),
       dinner + walking tour ($15+$25).
Day 2: Museum pass ($20), paid attraction ($40), meals ($50).
Day 3: Free morning, meals ($25), return bus (included above),
       checkout.
Total spent: $300. Remaining buffer: $200 for
souvenirs/contingency, staying safely under the $500 cap.
```

### Design Rationale

- Least-to-Most breaks one hard task into small, ordered sub-tasks so the model never has to solve everything at once.
- Each sub-answer (e.g., transport cost) becomes context for the next (e.g., lodging), which keeps the running budget accurate.
- The final combination step forces the model to check that the sum stays within $500, reducing arithmetic drift common in one-shot planning.

## Q13. Meta-Prompting (Prompt That Writes a Prompt)

*Question: Write a prompt whose job is to generate an optimized prompt for a downstream task — e.g., ask the model to design the best possible prompt for summarizing legal contracts for non-lawyers.*

### Prompt

```
You are a prompt engineering expert. Your job is NOT to summarize a
```

```
contract yourself. Instead, design the best possible prompt that a
downstream LLM could use to summarize legal contracts for
non-lawyer readers.

The prompt you design must:
1. Define a clear role (e.g., "You are a legal-to-plain-English
   translator").
2. Specify the audience (non-lawyers, no legal jargon).
3. Specify the required output format (e.g., bullet summary with
   sections: Parties, Obligations, Payment Terms, Risks,
   Termination).
4. Include an instruction to flag any clause that could be risky
   or unusual for the reader.
5. Include one short example (input clause -> plain-English
   output) to guide the model.

Output ONLY the final downstream prompt, ready to copy-paste.
```

### Simple Grok-LLM Command (for reference / execution)

```
curl https://api.x.ai/v1/chat/completions \
  -H "Authorization: Bearer $GROK_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
     "model": "grok-4",
    "messages": [
       {"role": "user", "content": "<paste the meta-prompt above>"}
    ]
  }'
```

### Sample Model Output — the Generated Downstream Prompt

```
You are a legal-to-plain-English translator. Your reader has no
legal training, so avoid jargon and explain any unavoidable legal
term in one short clause.

Read the contract text provided below and summarize it using
exactly these sections:
- Parties: who is involved and their role.
- Obligations: what each party must do.
- Payment Terms: amounts, due dates, penalties.
- Risks: anything unusual, one-sided, or that could hurt the
  reader, flagged with '⚠'.
- Termination: how and when the contract can end.

Example:
```

```
Input clause: 'Lessee shall indemnify and hold harmless Lessor
from any claims arising from Lessee's use of the premises.'
Output: 'If something goes wrong because of how you use the
property, you (not the landlord) are responsible for covering
any related claims or costs.'

Now summarize the following contract:
{{contract_text}}
```

### Design Rationale

- Meta-prompting shifts the model's job from doing the task to designing the instructions for the task, which improves reusability across many contracts.
- Naming the role, audience, format, and risk-flagging requirement up front removes ambiguity that would otherwise cause inconsistent summaries.
- Requesting one worked example inside the generated prompt gives the downstream model a concrete pattern to follow (few-shot anchoring).

# Q13 — Meta-Prompting (Prompt That Writes a Prompt)

**Task:** Write a prompt whose job is to generate an *optimized prompt* for a downstream task — summarizing legal contracts for non-lawyers.

This notebook:

1. Sends a **meta-prompt** to Grok, asking it to design the best possible summarization prompt.
2. Takes the **generated prompt** and uses it on a sample contract clause.
3. Documents technique selection, output effectiveness, and design rationale (per rubric).

Just run all cells top to bottom. You only need to paste your **xAI (Grok) API key** below.

## 0. Setup

```
!pip install -q requests

import requests
import json
import getpass

# Paste your xAI API key when prompted (input is hidden)
XAI_API_KEY = getpass.getpass("Enter your Grok (xAI) API key: ")

GROK_URL = "https://api.x.ai/v1/chat/completions"
GROK_MODEL = "grok-4"  # change to whichever Grok model you have access to, e.g. "grok-4-fast"

def call_grok(messages, model=GROK_MODEL, temperature=0.4, max_tokens=1200):
    headers = {
        "Authorization": f"Bearer {XAI_API_KEY}",
        "Content-Type": "application/json"
    }
    payload = {
        "model": model,
        "messages": messages,
        "temperature": temperature,
        "max_tokens": max_tokens
    }
    resp = requests.post(GROK_URL, headers=headers, data=json.dumps(payload))
    resp.raise_for_status()
    data = resp.json()
    return data["choices"][0]["message"]["content"]

print("Setup complete.")
```

## 1. The Meta-Prompt

This prompt does not summarize anything itself — it instructs the model to **design a prompt** for the downstream summarization task. This is the core meta-prompting technique.

```
META_PROMPT = """
ROLE: You are an expert prompt engineer specializing in legal-to-plain-language
communication tasks.

TASK: Design an optimized prompt that will be used by another LLM instance to
summarize legal contracts for non-lawyer readers (e.g., small business owners,
freelancers, tenants).

REQUIREMENTS FOR THE PROMPT YOU DESIGN:
1. Specify the ideal persona/role for the summarizing model to adopt.
2. Include explicit output structure (e.g., sections like "Key Obligations,"
   "Risks/Red Flags," "Deadlines," "Plain-English Summary").
3. Instruct the model to flag ambiguous or risky clauses rather than silently
   omitting them.
4. Embed a chain-of-thought instruction so the model reasons through each
   clause before summarizing it, without exposing that reasoning in the final
```

```
   output (i.e., "think step-by-step internally, then output only the final
   structured summary").
5. Include ONE worked example (one-shot) showing a short sample clause and its
   ideal plain-English translation, to anchor tone and simplicity level.
6. Add explicit constraints: reading level (8th-10th grade), no legal jargon
   without a plain-language gloss, length cap per section.
7. Include a safety/accuracy instruction: the model must not give legal advice,
   only explain what the contract says, and must recommend consulting a lawyer
   for decisions.

OUTPUT FORMAT:
Return ONLY the final prompt text, ready to copy-paste into another LLM. Do not
include explanations before or after it. Wrap it in triple backticks.
"""

messages = [{"role": "user", "content": META_PROMPT}]
generated_prompt_raw = call_grok(messages)

print(generated_prompt_raw)
```

## 2. Extract the Generated Prompt

Grok returns the optimized prompt wrapped in triple backticks. This cell pulls it out cleanly so it can be reused programmatically.

```
import re

def extract_code_block(text):
    match = re.search(r"```(?:\w*\n)?(.*?)```", text, re.DOTALL)
    return match.group(1).strip() if match else text.strip()

GENERATED_PROMPT = extract_code_block(generated_prompt_raw)
print(GENERATED_PROMPT)
```

## 3. Use the Generated Prompt on a Sample Contract Clause

Now we test the *downstream* prompt end-to-end: feed it a real (sample) lease clause and check output quality against the rubric (structure, safety, plain language).

```
SAMPLE_CONTRACT = """
SECTION 4 - INDEMNIFICATION
Tenant shall indemnify, defend, and hold harmless Landlord from and against any
and all claims, damages, losses, and expenses, including reasonable attorney's
fees, arising out of or resulting from Tenant's use or occupancy of the
Premises, regardless of whether such claim is caused in whole or in part by
the negligence of Landlord.

SECTION 7 - TERMINATION
Landlord may terminate this Agreement upon thirty (30) days written notice for
any reason or no reason, at Landlord's sole discretion.

SECTION 9 - LATE FEES
Rent not received by the 5th of the month shall incur a late fee equal to 10%
of the monthly rent, plus $25 per additional day of delinquency.
"""

final_prompt = GENERATED_PROMPT.replace("[INSERT CONTRACT TEXT]", SAMPLE_CONTRACT)

messages = [{"role": "user", "content": final_prompt}]
summary_output = call_grok(messages)

print(summary_output)
```

## 4. Prompt-Output Documentation

| Input | Output | Observed Effective |
|---|---|---|
| Meta-prompt (Section 1) | Optimized summarization prompt (Section 2) | Should return a structured, reusable prompt with persona, sections, one-shot examp |
| Generated prompt + sample lease clauses (Section 3) | Structured 5-section plain-English summary | Check: does it flag the indemnification clause as a red flag? Does it define "indemnit |

Run the cell below to auto-check a few of these signals in the actual output your API call produced.

```
checks = {
    "Mentions indemnify/indemnification": "indemni" in summary_output.lower(),
    "Defines jargon in plain terms": "means" in summary_output.lower() or "in other words" in summary_output.lower() or "(" in
    "Has a red-flag / risk section": "red flag" in summary_output.lower() or "risk" in summary_output.lower(),
    "Recommends consulting a lawyer (not giving advice itself)": "lawyer" in summary_output.lower() or "attorney" in summary_ou
    "Avoids directive advice language ('you should')": "you should" not in summary_output.lower()
}

for k, v in checks.items():
    print(f"{'PASS' if v else 'CHECK MANUALLY'} -- {k}")
```

## 5. Technique Selection Rationale

| Technique | Where applied | Why |
|---|---|---|
| **Zero-shot instruction** | Section headers requirement (#2) | Structure is unambiguous enough without an example |
| **One-shot example** | Requirement #5 (indemnification example) | Anchors tone/simplification level — hard to specify through instruction alone |
| **Hidden Chain-of-Thought** | Requirement #4 | Improves per-clause reasoning accuracy without cluttering the final output with visible reasoning |
| **Role/persona priming** | Requirement #1 | Shapes vocabulary and register toward plain-language explanation |
| **Constraint-based prompting** | Requirements #6, #7 | Keeps output consistent, safe, and reusable across different contracts |

## 6. Design Iterations (Notes)

- **v1**: "Write a prompt to summarize contracts" — too vague, generated prompt lacked structure and safety guardrails.
- **v2**: Added requirement #7 (no legal advice) after v1's generated prompt let the model imply recommendations.
- **v3 (final, used above)**: Added the hidden CoT instruction after noticing visible reasoning made downstream outputs too long/legalistic for non-lawyer readers.